

Project Report

Project Report – Scholar's Digital Library

This document accompanies the implemented code for the Online Library project. It covers the required explanations, design, testing, and an appendix describing the use of AI tools during development.

1. Brief theoretical derivation and approach

- Problem: build an online library (bookstore) storing metadata for media items: name, publication date, author, category. The application must provide backend endpoints and a GUI frontend.

- Approach: a lightweight REST backend (Flask) provides the HTTP endpoints required by the assignment. A desktop GUI client (Tkinter) acts as the frontend and requests data from the backend. Persistence is performed using a JSON file (`media.json`) to keep the implementation simple and easy to inspect.

2. Answers to assignment questions

- Functional requirements implemented:

- List all media items (GET `/media`).
- List media items by category (GET `/media?category=...`).
- Search for a media item by exact name (GET `/media/search?name=...`).
- Display metadata for a specific media item (GET `/media/<id>`).
- Create a new media item (POST `/media`).
- Delete a specific media item (DELETE `/media/<id>`).
- Update/edit a media item (PUT `/media/<id>`) – implemented to support the GUI edit feature.

- Non-functional considerations: code is modular (`storage.py`, `server.py`, GUI in `main.py`), uses UUIDs for stable IDs, includes basic input validation and error responses, and provides automated tests.

3. Description of main approaches in program code

- `storage.py`: central module handling JSON file I/O with a simple mutex (`threading.Lock`) and convenience functions: `list\_all`, `list\_by\_category`, `search\_by\_name\_exact`, `get\_by\_id`, `create\_item`, `update\_item`, `delete\_item`.

- `server.py`: Flask application exposing the HTTP endpoints used by the frontend.

- `main.py`: Tkinter GUI. The client prefers to call the backend endpoints (localhost:5000). If the backend is unavailable, it falls back to local JSON read/write so the GUI still works without starting the server.

4. Difficulties and how they were overcome

- Editing previously relied on matching visible text which is fragile; resolved by adding per-item UUIDs and using them as stable identifiers.

- Ensuring the GUI works whether or not the backend is running: implemented an HTTP-first approach with a local fallback.

5. Optimization steps applied

- Centralized data access in `storage.py` to avoid code duplication and make it testable.

- Added server endpoints and tests allowing CI-friendly validation of functionality.

6. Discussion and future improvements

- Strengths: meets assignment endpoints and frontend requirements; modular; includes tests and documentation.

- Weaknesses: JSON file is not suitable for concurrent multi-user scenarios; a DB (SQLite/Postgres) would be needed for production.

- Future work: authentication, borrowing/reservation states, returning items, pagination, packaging as a web frontend.

7. Tests included

- `tests/test\_storage.py` – tests create & delete operations for `storage.py` functions.

- `tests/test\_server.py` – tests the Flask endpoints using Flask's test client (create, list, search, delete).

8. AI tools appendix

- I used an AI coding assistant to help implement and refactor code. Prompts included requests to implement a Flask backend, storage module, tests, and GUI integration. All AI outputs were reviewed line-by-line and corrected where necessary. Below is a short summary of the AI usage (full conversation transcript and prompts should be included in the Appendix when submitting):

- AI produced scaffolding for `server.py` and `storage.py`.

- I verified and adjusted behavior to ensure stable IDs and safe persistence.

9. How to run and how to produce PDF submissions

1. Install dependencies:

```
```
```

```
pip install -r requirements.txt
```

```
```
```

2. Start the backend server (in a terminal):

```
```
```

```
python server.py
```

```
```
```

3. Start the GUI (in another terminal):

```
python main.py
```

To produce PDFs required by the assignment (print source files and report): open the files in an editor and use "Print to PDF" or use a command-line tool to convert Markdown/py files to PDF. The `docs/Project\_Report.md` file can be converted with tools like `pandoc`.